

Acknowledgements

"I would like to express my sincere gratitude to all those who have supported me throughout this project. Firstly, I would like to thank my supervisor and project lead Mr. Aman Gupta and the BQC team (Daniel & Venkat) for constantly providing me with invaluable guidance and helping me with my coding practices. I would also like to thank my internal guide Dr. Alka V. Deshmukh for providing constant support throughout the project. Finally, I would like to acknowledge the support of my family and friends in Pune as well as Mumbai, who have been a constant source of encouragement, motivation and Fun. Speical mention to my sister Pratibha Sharma, who has been guiding me all my life. Without the support of all these individuals, this project would not have been possible."

Shōganai

Dedicated to my Parents & Didi, for always
trusting me and my decisions.



Abstract

The introduction of Quantum Information Processing and proven quantum algorithms provide a speed boost over classical algorithms. Quantum computing has become reality, rather a noisy reality(cries in NISQ). With various industry leaders such as IBM, Google, etc. Invest heavily in quantum Research & development activities and collaborations with Quantum Research Institutes. Quantum computers are effectively hosted in the cloud and may be accessed through services like AWS Braket.

These cloud Units are constantly vulnerable to data breaches with the service provider. How does one ensure that the user's quantum simulations are secure? The widely used circuit-based model of quantum computing is well known. Yet, there is another kind of computation known as measurement-based quantum computing (MBQC). To do repeated projective measurements and perform the needed work, MBQC employs cluster states, which are highly entangled qubits. With a few changes, this MBQC approach can truly blind the server. The term "blinding the server" refers to the fact that the server will not understand the implemented Quantum gates and thus can be blinded.

Contents

Acknowledgements	i
Abstract	iii
I Quantum Computing	1
1 Introduction	2
1.1 Classical Computing	2
1.2 The Bit	3
1.3 Computational Bottleneck	4
2 Basics Of Quantum Computing	7
2.1 What & How?	7
2.2 The Qubit	8
2.2.1 Visualizing a Qubit	9
2.3 Quantum Gates	10
2.4 Quantum Algorithms	11
II Blind Quantum Computing	14
3 Introduction to Cluster States and Measurement Based Quantum Computing (MBQC)	15
3.1 Introduction & Motive	15
3.2 Measurement Based Quantum Computing (MBQC)	16
3.3 Cluster States	17

3.3.1	Quantum Gates as Cluster States	17
4	Universal Blind Quantum Computing (UBQC) Module Development	19
4.1	MBQC To UBQC	22
5	Conclusion & Future Scope	24

Part I

Quantum Computing

Chapter 1

Introduction

1.1 Classical Computing

Classical computing? What do I even mean when I mention classical computing? With the notion of quantum information science laptops, mobile phones other devices have started to be referred to as classical devices or classical computers. These devices follow the classical information theory and their computations are referred to as classical computing.

These devices are based on digital circuits such as gates, transistors, integrated circuits (ICs), and various other circuits. The bit is the basic data processing unit in these devices. Digital circuits are very efficient computationally and space compact. Supercomputers, cloud-based powerful computing machines can process billions of petabytes of data. The data can be interpreted, cleaned, and process further to make scientific inferences. One of the marvels of computing power is the EHT which is a large telescope array consisting of a global network of radio telescopes. The idea here is all the telescopes in the EHT point at the same radio source and then an algorithm stitches the data received from all the telescopes into one single image. The data recorded rounds up to 15 PB ($1Pb = 1 \times 10^{15}$) Byte over a year per telescope.

Even the biggest of these electronic devices are made up of the smallest transistors which function in the same way for all the devices. The smallest transistor available to humans is of the size of 5 nanometers. Moore's law is the observation that the number of transistors in an integrated circuit (ICs) doubles about every two years. Here classical computing poses a solid rigid wall of size compatibility as below the scales of nanometers enters the realm of quantum scales. At these scales the uncertainty principle is very evident and the laws of classical physics break down. Thus posing a challenge to the Moores' law.

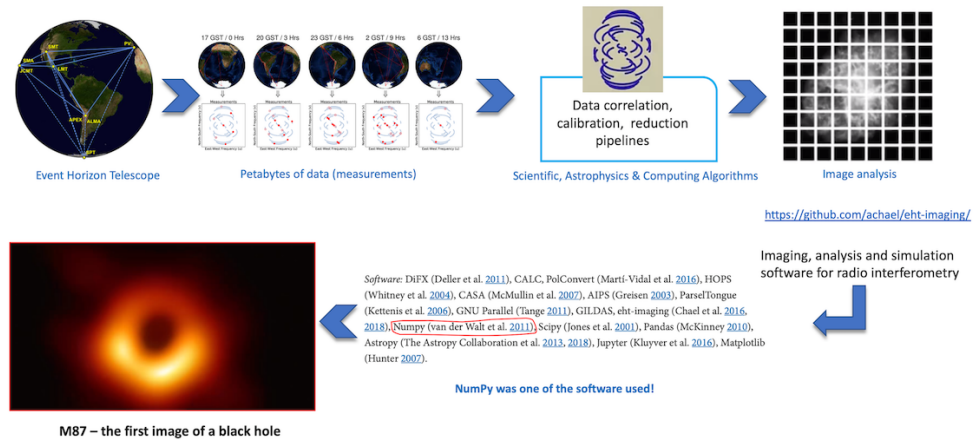


Figure 1.1: EHT Technique

1.2 The Bit

The Bit is the basic computational unit of the classical devices or classical computing described in the previous section. The bit is the most basic unit of information in computing and digital communications. The name is a shorthand for binary digit. A bit (binary digit) is the smallest unit of data that a computer can process and store. A bit is always in one of two physical states, similar to an on/off light switch. The state is represented by a single binary value, usually a 0 or 1.

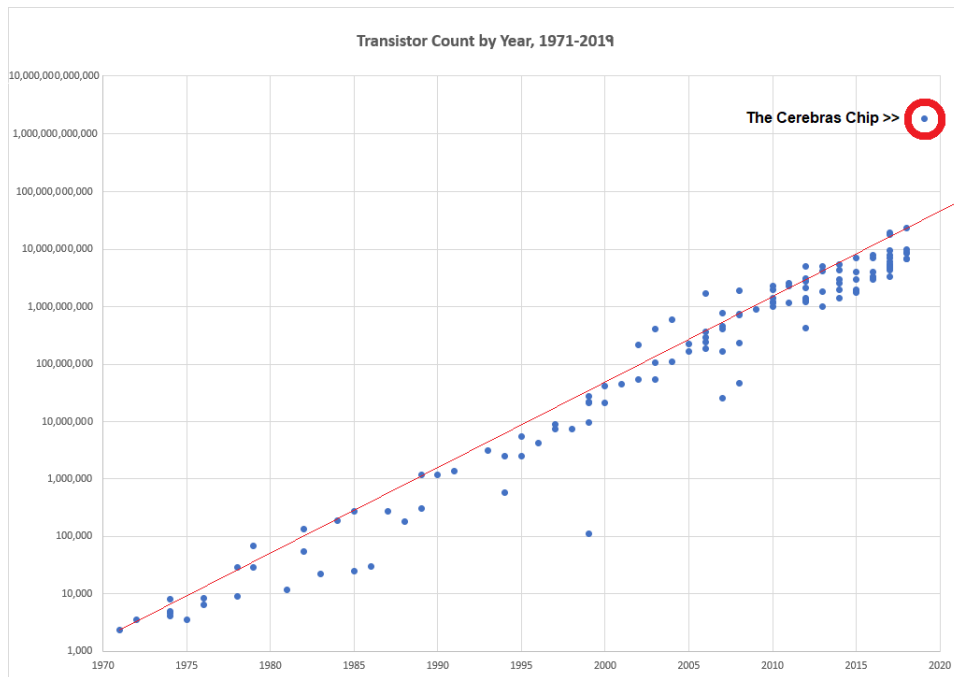
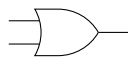


Figure 1.2: Moores' Law

Further manipulation of bits can be performed by using logic gates, these gates are the fundamental processing unit of any classical devices. A logic gate is an idealized or physical device that performs a Boolean function, a logical operation performed on one

or more binary inputs that produce a single binary output. The logic gates follow a set of rules on the basis of the input process output.

A few of the basic gates are AND, OR, and NOT and the truth table is as follows;



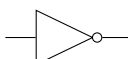
OR GATE

Bit 1	Bit 2	Output
0	0	0
0	1	1
1	0	1
1	1	1



AND GATE

Bit 1	Bit 2	Output
0	0	0
0	1	0
1	0	0
1	1	1



NOT GATE

Bit 1	Output
0	1
1	0

1.3 Computational Bottleneck

All seems well and good to the eyes of laymen, we have all we wish for right at our fingertips. Tools like ChatGPT-3 and 4, have achieved new levels of computational supremacy. Machine learning, Artificial Intelligence, and Deep Learning are on an all-time high in the past century. In computer science, we categorize problems according to how many computational steps it would take to solve them using the best-known

algorithms. That is translated to how long it will take a computer to solve them. These categories are broad and often overlap. The most used three categories are;

- P problems
- NP problems
- NP-complete

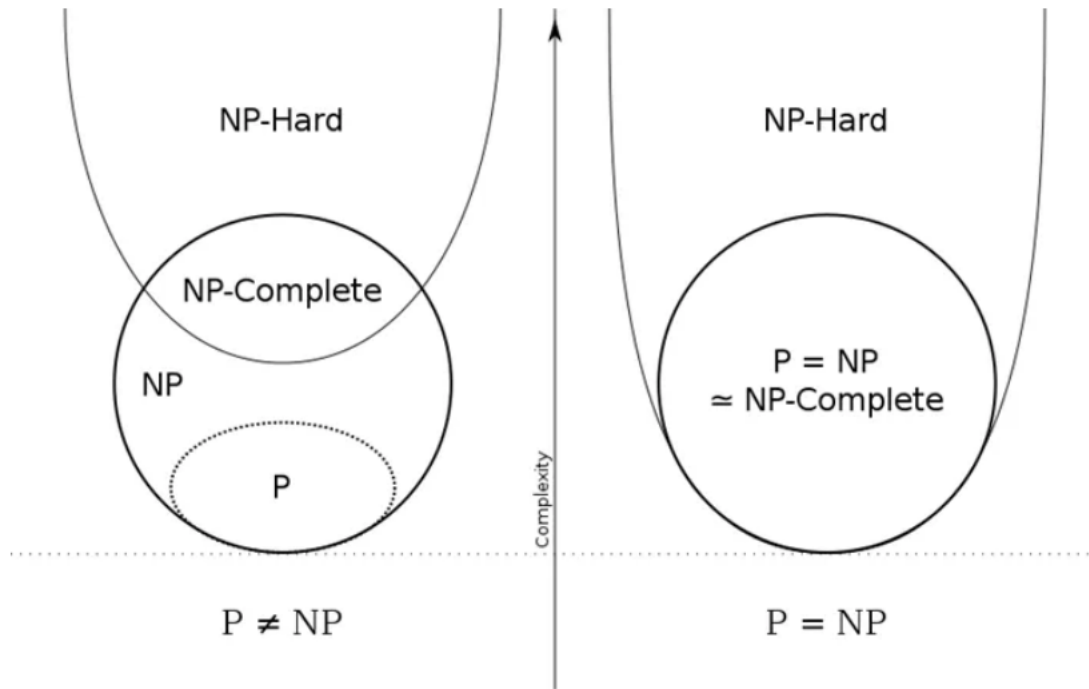


Figure 1.3: Complexities

Yet there exist problems that cannot be simulated on a computer only due to computational capacity hindrances [6]. Problems such as Data mining, cryptography, and protein folding problem are a few of the problems where even the best supercomputers fail in order to run these simulations accurately with low error rates. To make faster, better computers we need to make our basic unit (transistor) more efficient and faster, but we are already facing the microscale barrier of Quantum mechanics, furthermore crunching the transistors is not going to help us unless we build a device and information processing theory that works in those scales. This is one of the fundamental reasons for the origin of Quantum Computing and Quantum Information Theory. There exists a proposed class of complex problems where it is hypothesized that a Quantum Computer will outperform a classical computer. (Note: Here I would like to stress the point that Quantum Computers are an exciting new field of study but to relate them to solve problems of climate change, space exploration is not just stupid but also a factor of addition to unnecessary hype around the domain.)

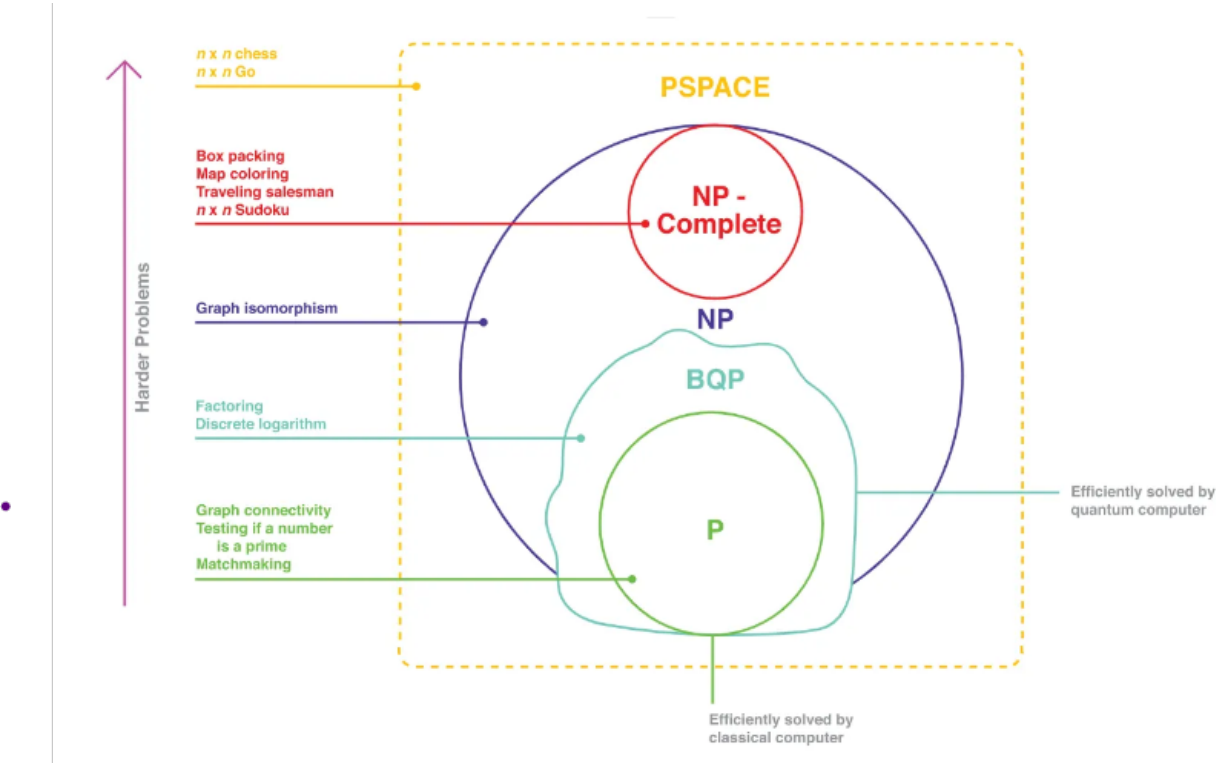


Figure 1.4: Quantum Complexities

Chapter 2

Basics Of Quantum Computing

2.1 What & How?

At the end of the previous chapter we saw one of the major reasons why Quantum Computing makes sense and should be not the focus right now but we need to have a small idea of it. In this chapter, I will reason with a few more reasons for quantum computing and explain What exactly it is and How one can get their head around this interdisciplinary domain of literally every branch of pure science.

Previously I mentioned how Moore's law states the computational power doubles every 2 years or so and now were at a point where going below 5nm, we are in the quantum regime where the laws of classical mechanics break down. One of the most prominent figures in Physics and early pioneer of quantum mechanics who also proposed the idea of quantum computers in the 1970s Richard Feynman said

"Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical, and by golly it's a wonderful problem, because it doesn't look so easy"

This statement alone was the reason for quantum computing's research to be initiated. The idea of quantum computing has been around since the 1970s. In the year 1992, two of the most promising quantum algorithms Deutsch-Jozsa algorithm [4] and Bernstein-Vazirani algorithm [?] came out which were mathematically proven algorithms. These suggested that given a quantum computer, one could use these algorithms and perform the tasks much faster than classical counterparts. Followed by Shor's [3] factoring algorithm in 1998 which suggested that the factoring of an n-digit number could be done exponentially faster using a quantum computer.

Many important problems in physics, especially low-temperature physics and many-body physics, remain poorly understood because the underlying quantum mechanics is

vastly complex. Conventional computers, including supercomputers, are inadequate for simulating quantum systems with as few as 30 particles because the dimension of the Hilbert space grows exponentially with particle number. Thus to simulate quantum systems efficiently one has to consider building a quantum computer.

2.2 The Qubit

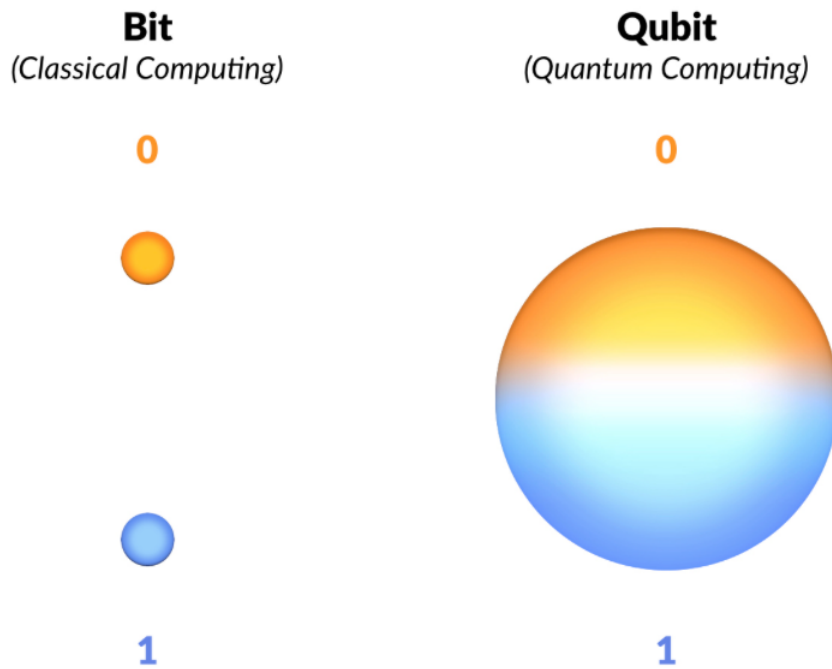


Figure 2.1: Bit & Qubit

Analogous to the Bit which is the fundamental unit of processing classical computing, for quantum computing we have something called a *Qubits* which is the fundamental unit of processing of a quantum computer. A qubit is quantum mechanical physical object that harnesses the quantum phenomena such as superposition, entanglement and interference. The two states of a qubit are often represented as $|0\rangle$ and $|1\rangle$, which are analogous to the classical 0 and 1. However, a qubit can also exist in a linear combination of these two states, represented as $\alpha|0\rangle + \beta|1\rangle$, where α and β are complex numbers and satisfy the normalization condition $|\alpha|^2 + |\beta|^2 = 1$. This property of qubits is the major advantage due to which we see an exponential speed up in terms of space. Consider we wish to transfer the number 588 using binary number system it translates to 1001001100, thus we can conclude that if we wish to process n bits of data we need n bits.

$$n \text{ bit data (1001001100)} \rightarrow n \text{ qubits (10 qubits)}$$

When we switch to qubits for information processing as a qubit is in a superposition of $|0\rangle$ and $|1\rangle$, with n qubits we can access 2^n computational states. One can see that with 2^n qubits we increase the accessible states exponentially.

$$n \text{ qubits} \rightarrow 2^n \text{ computational states}$$

A qubit can be physically realized using various systems, such as the spin of an electron or the polarization of a photon. The ability of qubits to exist in superpositions of states and to become entangled with other qubits is what gives quantum computers their power. By manipulating qubits through quantum gates, a quantum computer can perform calculations on many inputs simultaneously, which can lead to exponential speedup for certain types of problems compared to classical computers.

However, qubits are also highly susceptible to environmental disturbances and decoherence, which can cause errors in calculations.

2.2.1 Visualizing a Qubit

The Bloch sphere is a geometric representation of the state of a qubit, which provides a visual tool for understanding the behavior of quantum systems. The Bloch sphere is a sphere with the equator representing the state where the qubit has an equal probability of being in state $|0\rangle$ and $|1\rangle$ (i.e., a superposition state), and the north and south poles representing the pure states $|0\rangle$ and $|1\rangle$, respectively. The surface of the sphere represents all possible states of a qubit.

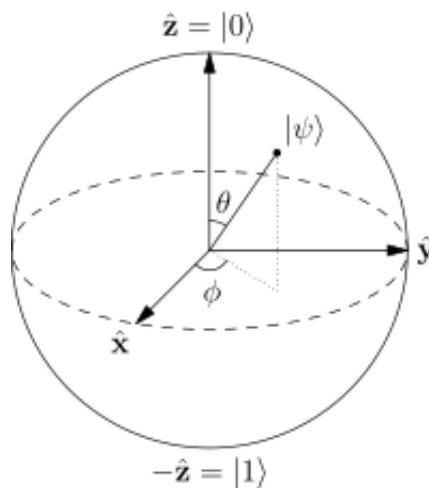


Figure 2.2: Bloch Sphere

The state of a qubit can be represented by a point on the surface of the sphere, and the distance of the point from the center of the sphere represents the magnitude of the coefficients α and β in the state vector $\alpha|0\rangle + \beta|1\rangle$. The angle that the point makes with the positive z -axis represents the relative phase between the two coefficients.

2.3 Quantum Gates

To manipulate quantum states in a Bloch sphere and perform computations on uses quantum logic gates or essential unitary operators. Quantum gates are mathematical operations that are applied to qubits in quantum computing in order to manipulate their state and perform quantum computations. Just as classical computers perform operations on classical bits, quantum gates operate on quantum bits or qubits.

There are many different types of quantum gates, each of which has a specific effect on the state of the qubits. For example, the Pauli-X gate (also known as the NOT gate) flips the state of a qubit from $|0\rangle$ to $|1\rangle$ or vice versa, while the Hadamard gate creates a superposition state of $|0\rangle$ and $|1\rangle$. Other gates include the CNOT gate (controlled NOT), which performs a NOT operation on one qubit depending on the state of another qubit, and the phase gate, which adds a relative phase to the state of a qubit.

Quantum gates can be combined in various ways to perform complex operations on qubits, similar to how classical gates can be combined to perform more complex operations on classical bits. For example, a quantum circuit is a series of quantum gates applied to one or more qubits in a specific sequence to perform a specific computation.

Quantum gates are represented by matrices as these gates are mathematical operations that act on quantum states, and matrices provide a convenient and efficient way to represent linear transformations.

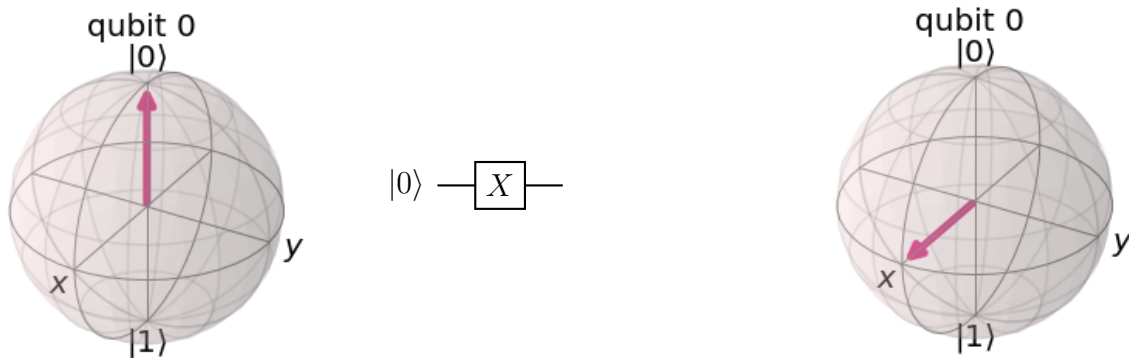


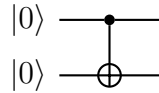
Figure 2.3: H-Gate

In quantum computing, the state of a qubit can be represented as a complex vector in a two-dimensional Hilbert space. Quantum gates are designed to operate on these vectors and transform them into new states. Mathematically, these transformations can be represented by linear operators, which can be represented by matrices. H-Gate is represented as

$$|0\rangle \text{ — } [H] \text{ — }$$

$$\text{Hadamard} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

The CNOT gate is a two-qubit gate that applies a NOT gate (equivalent to an X gate) to the second qubit if and only if the first qubit is in the state $|1\rangle$. This gate is represented by a 4x4 matrix, which describes how the gate transforms the state of the two qubits. The CNOT Gate looks as shown below



$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

2.4 Quantum Algorithms

Quantum algorithms are at the foundation of quantum computing, giving a collection of methods to perform on quantum computers to address specific computational problems. In this section, we will look at the fundamentals of quantum algorithms, their advantages over classical algorithms, and some of the most well-known quantum algorithms.

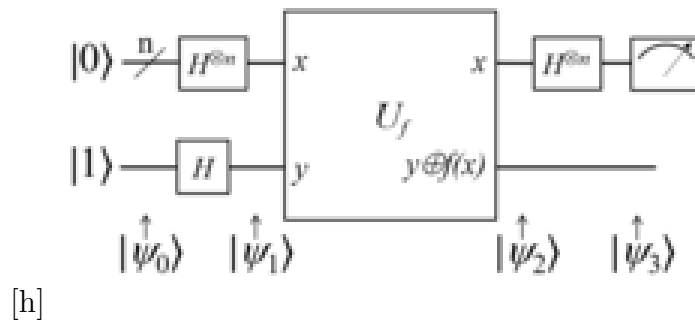


Figure 2.4: Quantum Circuit for Deutsch–Jozsa Algorithm

The principles of quantum algorithms are based on quantum physics phenomena such as superposition and entanglement. Superposition allows a qubit to be in multiple states at the same time, whereas entanglement allows for the correlation of qubits that is not possible with classical bits. These features enable quantum algorithms to accomplish computations that are intractable for classical algorithms. Quantum algorithms are meant to operate on quantum states and alter these states using quantum gates. Quantum gates

are mathematical procedures that work on qubits to change their states (Linear Transformations). In general, quantum algorithms are made up of a succession of quantum gates that are used to execute computation on a set of qubits. One of the primary benefits is the possibility of great computing speedup. For specific situations, quantum algorithms can provide an exponential speedup over classical algorithms, which means they can solve problems substantially faster. This is due to quantum mechanics' features, which allow quantum algorithms to conduct numerous calculations in parallel, allowing for quicker computation.

Several well-known quantum algorithms have been developed over the years. These algorithms solve a variety of problems in different areas. Here are a few examples:

- Grover's Algorithm - Grover's algorithm is a quantum algorithm for searching an unsorted database. It provides a quadratic speedup over classical search algorithms, meaning that it can search through a database of N items in roughly $O(\sqrt{N})$ time.

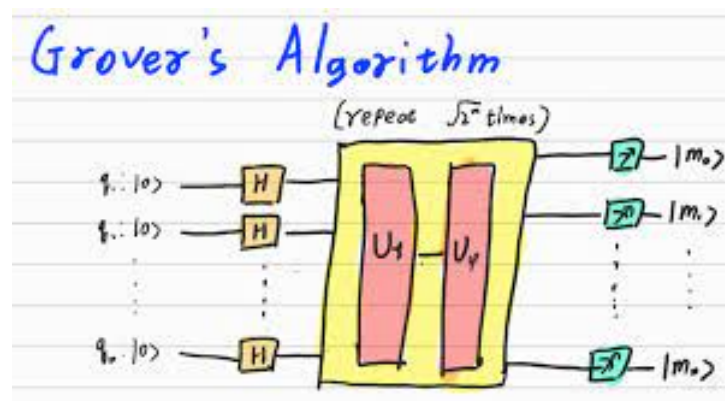


Figure 2.5: Grover's Algorithm (Unstructured Database Search)

- Shor's Algorithm - Shor's algorithm is a quantum algorithm for factoring large numbers into their prime factors. It provides an exponential speedup over classical algorithms, meaning that it can factor large numbers much faster than classical algorithms.

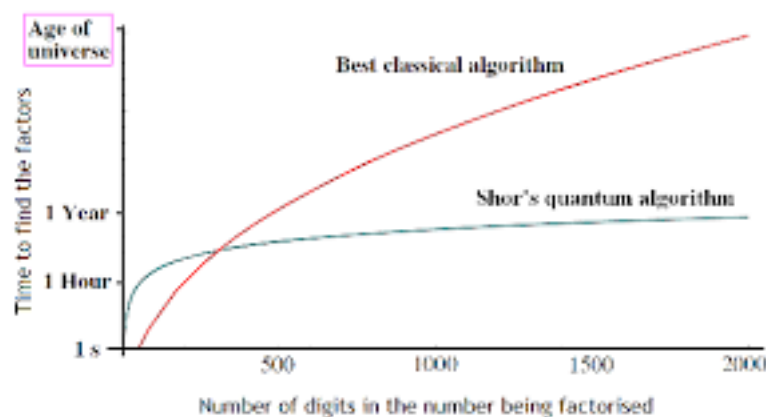


Figure 2.6: Shor's Algorithm, (Factoring of Numbers)

Quantum algorithms are a critical component of quantum computing, providing a collection of procedures that may be executed on quantum computers to address specific computational problems. They use quantum mechanics features to accomplish computations that are intractable for classical algorithms. The benefits of quantum algorithms include the potential for tremendous processing speedup and the ability to address problems that traditional algorithms cannot answer. Over the years, several well-known quantum algorithms have been developed, each with its own set of uses and advantages.

Part II

Blind Quantum Computing

Chapter 3

Introduction to Cluster States and Measurement Based Quantum Computing (MBQC)

3.1 Introduction & Motive

Blind Quantum Computing (BQC) refers to a set of protocols through which a client with ‘limited’ or ‘no’ quantum resources can perform the quantum operation on a ‘blind’ quantum server; the server is blind in the sense that the server has no information about the task that it is performing. The goal of BQC is to provide a way for clients to use quantum computing resources without requiring them to have their own quantum hardware or expertise. This is particularly important because quantum hardware is currently expensive and difficult to build and maintain, and quantum expertise is relatively rare.

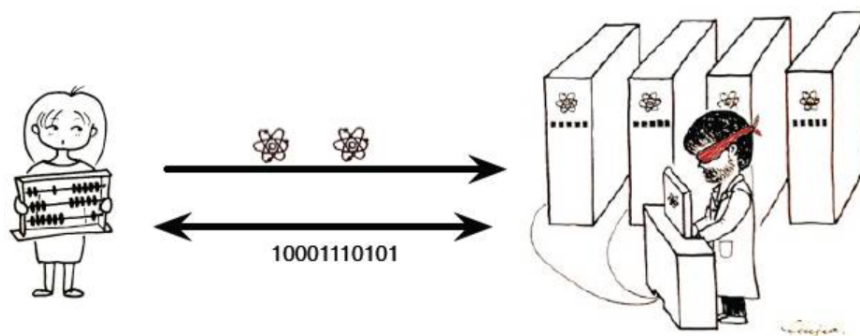


Figure 3.1: BQC

One of the key challenges in BQC is to ensure that the client’s computation remains private, even in the presence of a malicious server. There are several proposed implementations of BQC, including those based on optical systems and superconducting circuits

like the one being used in this project is the IBM's superconducting devices.

3.2 Measurement Based Quantum Computing (MBQC)

The most commonly used model of quantum computation is the quantum circuit model. Measurement-based quantum computation is a model for quantum computation in which all the basic dynamical operations are non-unitary quantum measurements, yet they can still be used to simulate arbitrary quantum dynamics as well as unitary dynamics.

MBQC is a model of quantum computing in which the computation is performed by a sequence of measurements on a highly entangled quantum state known as a cluster state (discussed in the next section). The computation is specified by a sequence of measurement operators, which are applied in a predetermined order. The output of each measurement is used as the input to the next measurement until the final output is obtained.

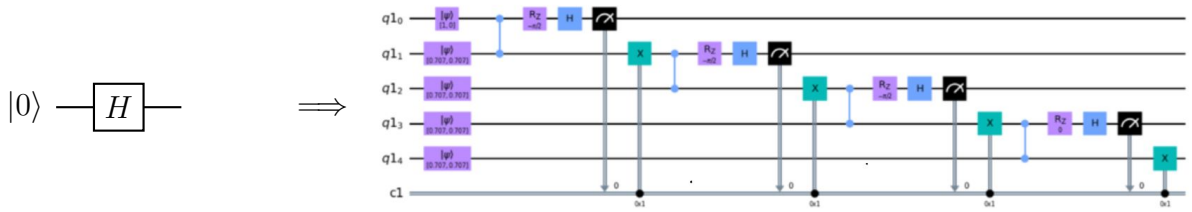


Figure 3.2: Circuit Model to MBQC

Although MBQC is equivalent to the circuit model of computation, there is very little work in the literature which deals with algorithms or protocols based on MBQC. One needs to develop the intuition which helps them to convert the circuit model into MBQC and vice-versa. It is also an intermediary step in the direction of using MBQC as a tool for Blind Quantum Computing. In MBQC, the quantum information is processed by a series of single-qubit measurements, which helps establish universal quantum computation.

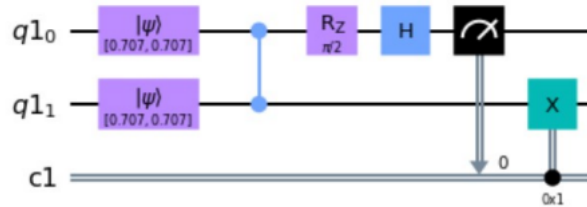


Figure 3.3: One Qubit Teleportation

The foundation of MBQC is the concept of one-bit teleportation. The circuit for the teleportation is shown in Fig. 1. One-bit teleportation is the building block of MBQC

where the measurements transform the corresponding input state into the desired output state. The first qubit can be measured in the (X, Y) plane - Hadamard gate on the second qubit can be applied after $U(\theta)$, and depending on the measurement outcome, an X Pauli correction is applied to the target qubit. The scheme is demonstrated on IBM qiskit using the state vector simulator.

3.3 Cluster States

Cluster states are a specific type of entangled quantum state that play a fundamental role in quantum computing and quantum information processing. In a cluster state, a group of qubits are all entangled with each other in a specific way so that the state of any one qubit depends on the states of all the others. The resulting entangled state is a cluster state, which has certain unique properties that make it useful for quantum information processing.

In MBQC, the system is prepared in a highly entangled quantum state called cluster states. Mathematical graphs efficiently parametrize such highly entangled states. A graph is an ordered pair $G = (V, E)$ where V is a set of N vertices and E is the set of edges $E \subseteq [V] \times [V]$ connecting the vertices in an arbitrary way. A cluster state can be associated to any graph G on n vertices. An n -qubit cluster state maybe defined in the following way:

- Each vertex corresponds to a qubit. Graph on n vertices correspond to a cluster state of n qubits.
- The qubits are prepared in the state $|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$ which is the $+1$ eigenstate of σ_x .
- Two-qubit phase gates or controlled-Z gates $U_{CZ} = \text{diag}(1, 1, 1, -1)$ is applied between all pairs of qubits connected by an edge.

Simulating a cluster state, as is into its circuit equivalent, is a very complex task and requires a large amount of memory. We identified a few key elements which are causing the overhead and are trying to optimize these issues.

3.3.1 Quantum Gates as Cluster States

One of the most important properties of cluster states is that they can be used to implement a universal set of quantum gates, which can be used to perform any quantum computation. This is because any quantum gate can be decomposed into a sequence of single-qubit gates and a CNOT gate, and a CNOT gate can be implemented using a cluster state.

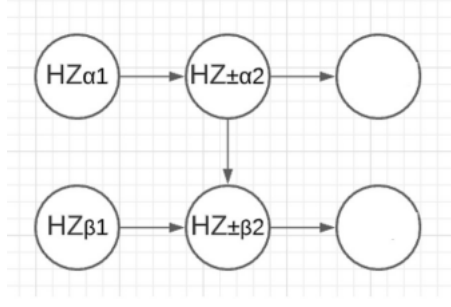
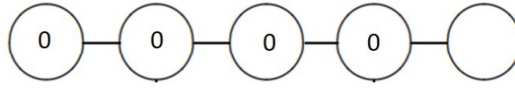
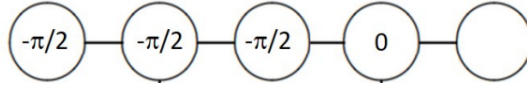


Figure 3.4: Cluster State

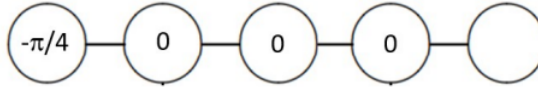
- A cluster state representation of the Identity gate can be:



- A cluster state representation of the Hadamard gate can be:



- A cluster state representation of the T gate can be:



Above shown are just a few of the cluster states for available and commonly used quantum gates, one can combine these cluster states to print and equivalent cluster state of multiple quantum gate by following this identity.

$$\begin{aligned}
 u(\theta, \phi, \psi) &= R_z(-\phi)R_x(-\theta)R_z(-\psi) \\
 &= R_z(-\phi)HR_z(-\theta)HR_z(-\psi) \\
 &= HHR_z(-\phi)HR_z(-\theta)HR_z(-\psi) \\
 &= HR_z(0)HR_z(-\phi)HR_z(-\theta)HR_z(-\psi)
 \end{aligned}$$

Figure 3.5: Cluster State Identity

where (θ, ϕ, ψ) are the euler angles for the corresponding gate or set of gates.

Chapter 4

Universal Blind Quantum Computing (UBQC) Module Development

Now we have the sufficient amount of knowledge and understanding to start moving from Measurement based quantum computing (MBQC) as our building block to the Blind quantum computing, we in a sense tweak the code in minimal manner and add random rotations to blind the quantum server.

The below code generates the **Euler angle** discussed previously for a single gate or set of gate(s).

```
#parameter for any arbitrary unitary gate
def euler_SU2(unitary):
    """
    unitary : Unitary U (2) to be converted to SU (2)
    """
    polar = cmath.polar
    acos = cmath.acos
    det = np.linalg.det(unitary) # Determinant function for 2 X 2
    # Parameterization
    mod_a = polar(g[0][0])[0]
    arg_a = polar(g[0][0])[1]
    arg_b = polar(g[0][1])[1]
    theta = 2*acos(mod_a).real
    phi = ((2*(arg_a + arg_b ) - np.pi)/2)
    psi = ((2*(arg_a - arg_b ) + np.pi)/2)
    euler_angles = [psi, theta, phi]
```

```
return euler_angles
```

Output:

```
: print("parameter for X gate",euler_SU2(np.array([[0,1],[1,0]])))
X_gate = euler_SU2(np.array([[1,0],[0,1]]))
parameter for X gate [3.141592653589793, 3.141592653589793, -3.141592653589793]
```

These angles can be further passed to a UBQC module developer code which output a desired UBQC quantum circuit. This circuit can be run on a quantum simulator and we receive a output as expected.

The following section of code does exactly that,

```
def pbyp(im, op, angle, u):

    iteration = angle
    # get euler angles for the given unitary
    parameters = u
    parameters.append(0)
    qubits = len(parameters)
    qreg = QuantumRegister(qubits+1)
    creg = ClassicalRegister(1)
    for p in range(len(iteration)):
        cluster = QuantumCircuit(qreg, creg)
        cluster.initialize(im, op-qubits)
        init = [1/sqrt(2), 1/sqrt(2)]
        for i in range(1, qubits+1):
            cluster.h(i)
        for j in range(qubits):
            cluster.rz(iteration[p], j)
        for x in range(qubits):
            cluster.cz(x, x+1)
            cluster.rz(-iteration[p], x)
```

```

cluster.rz(-parameters[x], x)
cluster.h(x)
cluster.measure(x,0)
cluster.x(qreg[x+1]).c_if(creg,1)
cluster.barrier()
cluster.draw('mpl')
backend = Aer.get_backend('qasm_simulator')
cluster.save_statevector()
display(plot_bloch_multivector(backend.run(
    cluster).result().
    get_statevector()))
return cluster, cluster.draw('mpl')

```

Output:

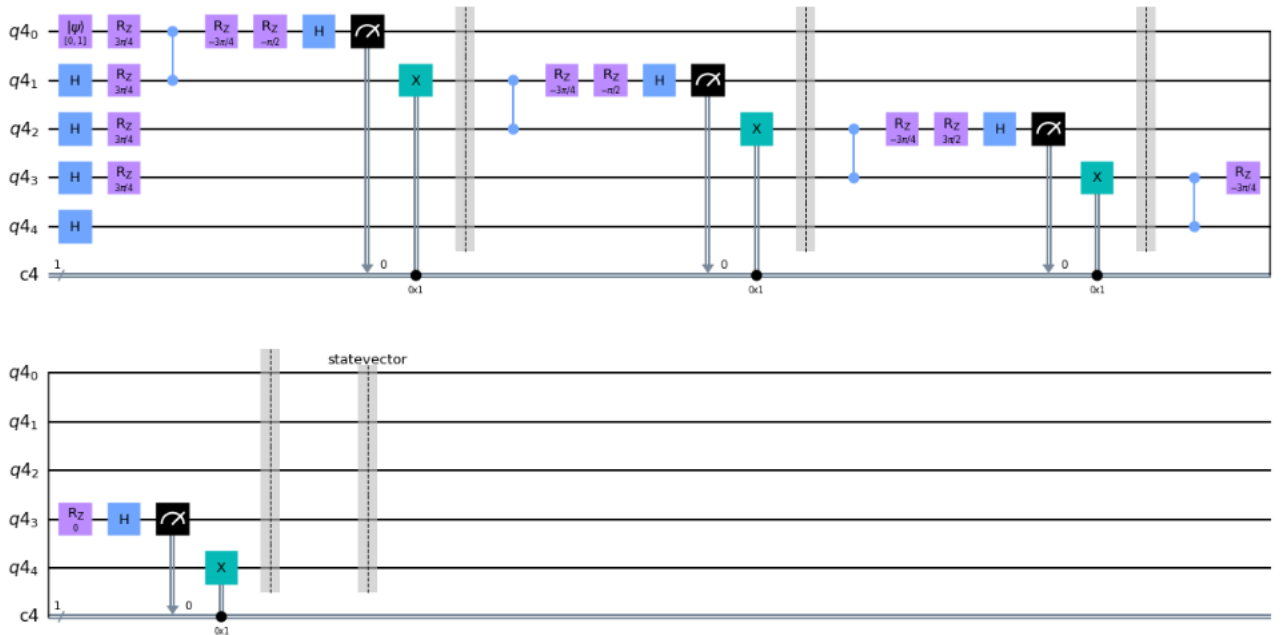


Figure 4.1: UBQC Circuit for H-Gate

4.1 MBQC To UBQC

The architecture switching from MBQC to UBQC involves multiple techniques. As the MBQC architecture is an alternate implementation of the Euler angles of any quantum gate, one of the ways to blind the quantum server is the introduction of random rotations with the respective Euler angle.

Thus under the MBQC architecture one applies the first Euler angle and then projects the computation on the second qubit, the qubits have been entangled previously. As the MBQC quantum circuit uses the $|+\rangle$ state as the initial state for computation to switch to UBQC we switch to a $|+\rangle_\theta$ initial state and compensate the added angle θ by subtracting it from the Euler angle thus blinding the server.

There are 2 major changes while switching from MBQC to UBQC the equations below encapsulate the idea,

$$\begin{array}{ccc} |+\rangle & \xrightarrow{\quad\quad\quad} & |+\theta\rangle = (|0\rangle + e^{i\theta}|1\rangle)/\sqrt{2} \\ (\phi') & \xrightarrow{\quad\quad\quad} & (\phi' - \theta) \end{array}$$

Figure 4.2: MBQC To UBQC

Here is the general circuit for the developing any universal blind quantum computing quantum circuit using measurement based quantum computing.

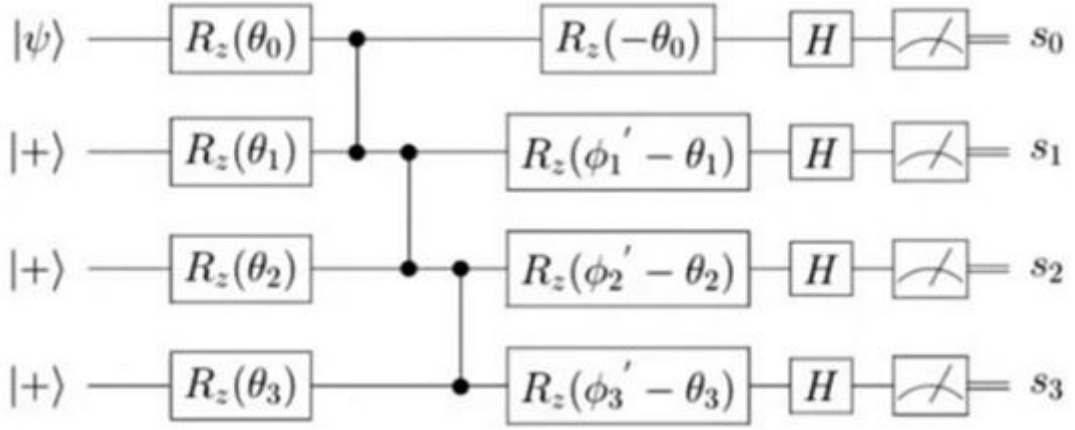


Figure 4.3: Genreal Idea of UBQC

Chapter 5

Conclusion & Future Scope

We can conclude that MBQC and one qubit teleportation technique have successful implementations for simulating a BQC circuit. We have successfully implemented 1D and 2D quantum gates using BQC. Further work in this domain directs to development of simulator for real backend. Also, we look forward to development of a BQC python library which can be open sourced for contribution and usage. Further algorithmic development includes development of new cryptographic algorithms such as Quantum Homomorphic encryption (QHE).

Bibliography

- [1] Michael A Nielsen and Isaac Chuang. Quantum computation and quantum information, 2002.
- [2] L. K. Grover, "A fast quantum mechanical algorithm for database search" In. Proceedings of the 28th Annual ACM Symposium on the Theory of Computing (STOC 1996), doi:10.1145/237814.237866,url: arXiv:quant-ph/9605043
- [3] Peter W. Shor "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer" url: <https://arxiv.org/abs/quant-ph/9508027>
- [4] David Collins, K. W. Kim, W. C. Holton "Deutsch-Jozsa algorithm as a test of quantum computation " doi: <https://doi.org/10.48550/arXiv.quant-ph/9807012> url: <https://arxiv.org/abs/quant-ph/9807012>
- [5] "What QC, Why QC, Who QC?" url: <https://medium.com/quantum-untangled/what-qc-why-qc-who-qc-4d5f601285e5>
- [6] "Strolling the Quantum Lane" url: <https://medium.com/quantum-untangled/strolling-the-quantum-lane-f8bad5eb85ef>
- [7] Anne Broadbent, Joseph Fitzsimons, Elham Kashefi "Universal Blind Quantum Computation " url:<https://arxiv.org/abs/0807.4154>
- [8] Shikhar's Report on MBQC (Qulabs)

- [9] "Event Horizon Telescope" url: <https://eventhorizontelescope.org/>
- [10] "IBM Qiskit" url: <https://qiskit.org/>